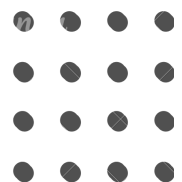




VMES Grinds

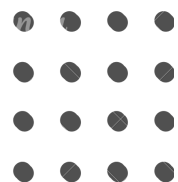




VMES Grinds

Pilot Program - Week 5 - Day 1

Stack





What is a Stack?





- A **linear** data structure
- **LIFO** (Last In, First Out)
- The **last element added** is the **first one removed**.



**A stack of
books**



**A pile of
plates**



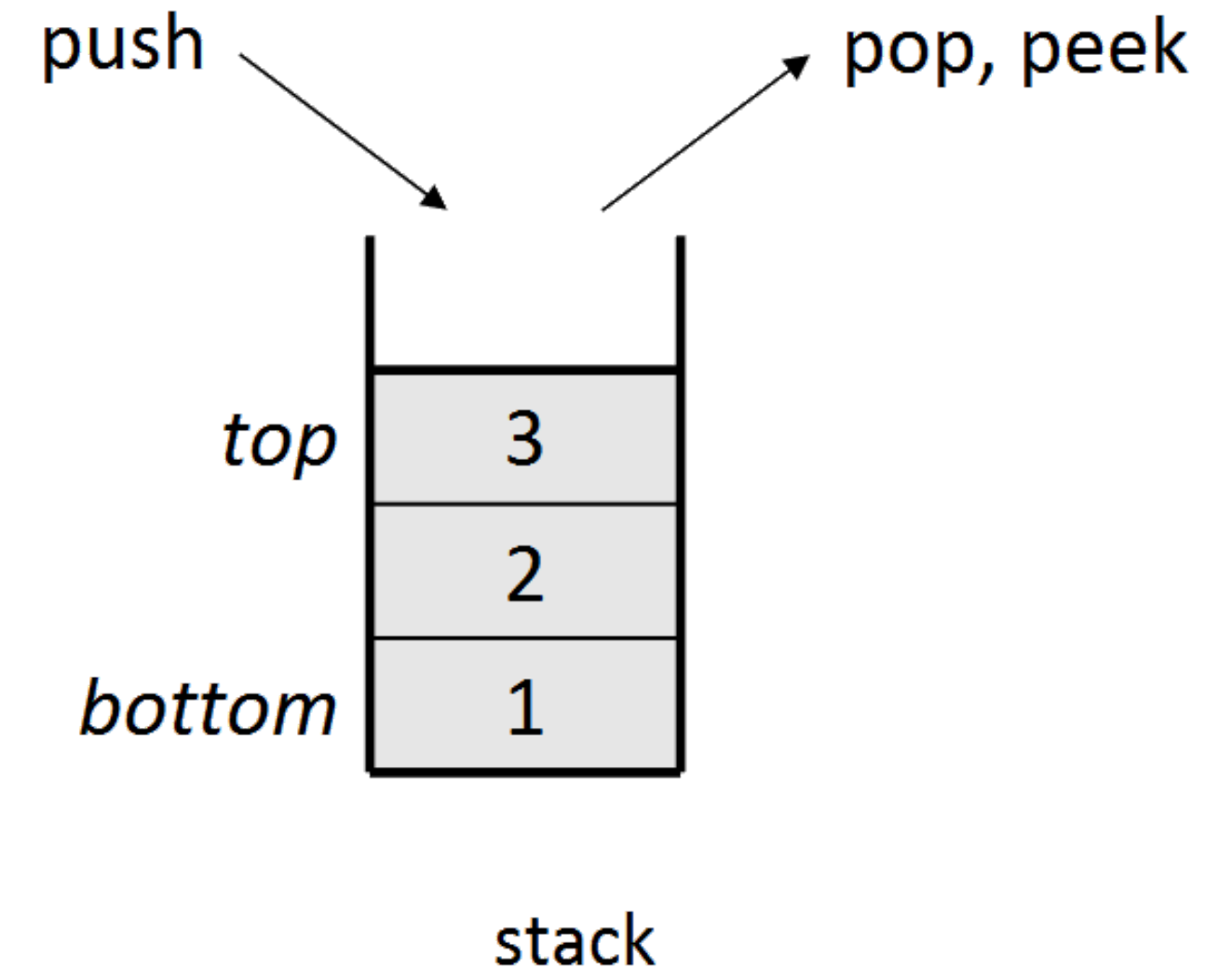
**A stack of
dvd/cd**

Fig: Realtime examples of Stack



Primary operations

- **push(x)** – add an item (x) to the top
- **pop()** – remove & return the top item
- **peek()** – return the top item without removing





Stacks can be implemented using

- **Array**
 - **Fixed size**
 - **Fast access**
- **Linked List**
 - **Dynamic memory**
 - **No overflow** unless system memory is full



Time Complexity → push, pop, peek → $O(1)$

Space Complexity → $O(n)$

Applications

- **Function Calls** in programming (recursion)
- **Expression Evaluation**
 - Postfix evaluation
 - Prefix/Postfix conversion
 - Balanced Parentheses Checking
- **Undo/Redo** Operations used in editing tools
- Browser **Back/Forward Navigation**
- Depth-First Search (**DFS**)
- **Backtracking** Algorithms
- **Maze solving**
- **N-Queens**



Advantages

- **Simple and easy** to implement
- **Fast operations** ($O(1)$)
- **Useful in many algorithms**

Disadvantages

- **Limited access**—can only access top element
- **Array-based stacks have fixed size**
- **Not suitable for random access**



20. Valid Parentheses





20. Valid Parentheses

Easy

Topics

Companies

Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Constraints:

- $1 \leq s.length \leq 10^4$
- `s` consists of parentheses only `'()[]{}'`.



Example 1:

Input: `s = "()"`

Output: `true`

Example 2:

Input: `s = "() [] {}"`

Output: `true`

Example 3:

Input: `s = "]"`

Output: `false`

Example 4:

Input: `s = "( [])"`

Output: `true`

Example 5:

Input: `s = "( [])]"`

Output: `false`



Explanation





Thought Process

- **Linear Iteration** → to check every character
- **Use Stack** → to check validity
- **Use dictionary** → for look up → **$O(1)$**



Optimized Code





```
class Solution:
    def isValid(self, s: str) -> bool:
        check = {'(': ')', '[': ']', '{': '}'
        stack = []

        for ch in s:
            if ch in check:
                if len(stack) == 0 or stack.pop() != check[ch]:
                    return False
            else:
                stack.append(ch)

        return len(stack) == 0
```



Time and Space Complexity Analysis





```
class Solution:
    def isValid(self, s: str) -> bool:
        check = {'(': ')', '[': ']', '{': '}' }
        stack = [] ← Space - O(n)

        for ch in s: ← Time - O(n)
            if ch in check:
                if len(stack) == 0 or stack.pop() != check[ch]:
                    return False
            else:
                stack.append(ch)

        return len(stack) == 0
```



Key Takeaway





- **Using dictionary is a good choice to lookup elements.**
 - **Key – Value make it easy to check matching.**
- **Recognizing the practical applications of DSA (Stack in this case) is really helpful in solving Leetcode problems (Parentheses checking in this case).**



VMES Grinds



Thank You !